```julia
using Markdown
```

```julia
using InteractiveUtils
```

Project 02 - Dual Slider Multibody Kinematics

System Description

- **Body 1**: Piston sliding on -45° track
- **Body 2**: Piston sliding on +45° track
- **Body 3**: Rigid bar (L = 10 cm), rotating at $\dot{\theta}_3$ = 2 rad/s, ends pinned to piston centers

Generalized Coordinates

$q = [x_1, y_1, \theta_1, x_2, y_2, \theta_2, x_3, y_3, \theta_3]$ (9 unknowns)

Constraint Equations C(q, t) = 0

1. $C_1 = y_1 - x_1$ (piston 1 on -45° track)
2. $C_2 = y_2 + x_2$ (piston 2 on +45° track)
3. $C_3 = \theta_1 + \pi/4$ (piston 1 orientation fixed)
4. $C_4 = \theta_2 - \pi/4$ (piston 2 orientation fixed)
5. $C_5 = x_3 - L/2 \cdot \cos(\theta_3) - x_1$ (pin joint: left end of bar to piston 1, x)
6. $C_6 = y_3 - L/2 \cdot \sin(\theta_3) - y_1$ (pin joint: left end of bar to piston 1, y)
7. $C_7 = x_3 + L/2 \cdot \cos(\theta_3) - x_2$ (pin joint: right end of bar to piston 2, x)
8. $C_8 = y_3 + L/2 \cdot \sin(\theta_3) - y_2$ (pin joint: right end of bar to piston 2, y)
9. $C_9 = \theta_3 - 2t$ (driven rotation at 2 rad/s)

I believe that it is possible to describe the motion of this system with only 6 contraint equations, but I wanted to use 9 to build understanding, and I think that the equations are simpler, non of this cos + sin

bs.

```
md"""
# Project 02 - Dual Slider Multibody Kinematics

## System Description
- **Body 1**: Piston sliding on -45° track
- **Body 2**: Piston sliding on +45° track
- **Body 3**: Rigid bar (L = 10 cm), rotating at θ̇₃ = 2 rad/s, ends pinned to piston centers

## Generalized Coordinates
q = [x₁, y₁, θ₁, x₂, y₂, θ₂, x₃, y₃, θ₃]  (9 unknowns)

## Constraint Equations C(q, t) = 0
1. C₁ = y₁ - x₁                          (piston 1 on -45° track)
2. C₂ = y₂ + x₂                          (piston 2 on +45° track)
3. C₃ = θ₁ + π/4                         (piston 1 orientation fixed)
4. C₄ = θ₂ - π/4                         (piston 2 orientation fixed)
5. C₅ = x₃ - L/2·cos(θ₃) - x₁           (pin joint: left end of bar to piston 1, x)
6. C₆ = y₃ - L/2·sin(θ₃) - y₁           (pin joint: left end of bar to piston 1, y)
7. C₇ = x₃ + L/2·cos(θ₃) - x₂           (pin joint: right end of bar to piston 2, x)
8. C₈ = y₃ + L/2·sin(θ₃) - y₂           (pin joint: right end of bar to piston 2, y)
9. C₉ = θ₃ - 2t                          (driven rotation at 2 rad/s)

I believe that it is possible to describe the motion of this system with only 6 contraint equations, but I wanted to use 9 to build understanding, and I think that the equations are simpler, non of this cos + sin bs.
"""
```

```
using NonlinearSolve, Plots, LinearAlgebra
```

C (generic function with 1 method)

```julia
begin
    # Constants
    L = 0.10  # bar length in meters (10 cm)

    # Constraint equations C(q, t) = 0
    # q = [x1, y1, θ1, x2, y2, θ2, x3, y3, θ3]
    function C(q, t)
        x1, y1, θ1, x2, y2, θ2, x3, y3, θ3 = q
        [
            y1 - x1,                          # C1: piston 1 on -45° track
            y2 + x2,                          # C2: piston 2 on +45° track
            θ1 + π/4,                         # C3: piston 1 orientation fixed
            θ2 - π/4,                         # C4: piston 2 orientation fixed
            x3 - L/2*cos(θ3) - x1,            # C5: left pin joint (x)
            y3 - L/2*sin(θ3) - y1,            # C6: left pin joint (y)
            x3 + L/2*cos(θ3) - x2,            # C7: right pin joint (x)
            y3 + L/2*sin(θ3) - y2,            # C8: right pin joint (y)
            θ3 - 2*t                          # C9: driven rotation
        ]
    end
end
```

Jacobian Cq = ∂C/∂q (9×9 matrix)

This was derived analytically by hand(impressive right) and is in my notes:

Row 1: [-1, 1, 0, 0, 0, 0, 0, 0, 0] Row 2: [0, 0, 0, 1, 1, 0, 0, 0, 0] Row 3: [0, 0, 1, 0, 0, 0, 0, 0, 0] Row 4: [0, 0, 0, 0, 0, 1, 0, 0, 0] Row 5: [-1, 0, 0, 0, 0, 0, 1, 0, L/2·sin(θ_3)] Row 6: [0, -1, 0, 0, 0, 0, 0, 1, -L/2·cos(θ_3)] Row 7: [0, 0, 0, -1, 0, 0, 1, 0, -L/2·sin(θ_3)] Row 8: [0, 0, 0, 0, -1, 0, 0, 1, L/2·cos(θ_3)] Row 9: [0, 0, 0, 0, 0, 0, 0, 0, 1]

```
md"""
## Jacobian Cq = ∂C/∂q  (9×9 matrix)

This was derived analytically by hand(impressive right) and is in my notes:

Row 1: [-1,  1,  0,  0,  0,  0,  0,  0,  0]
Row 2: [ 0,  0,  0,  1,  1,  0,  0,  0,  0]
Row 3: [ 0,  0,  1,  0,  0,  0,  0,  0,  0]
Row 4: [ 0,  0,  0,  0,  0,  1,  0,  0,  0]
Row 5: [-1,  0,  0,  0,  0,  0,  1,  0,  L/2·sin(θ₃)]
Row 6: [ 0, -1,  0,  0,  0,  0,  0,  1, -L/2·cos(θ₃)]
Row 7: [ 0,  0,  0, -1,  0,  0,  1,  0, -L/2·sin(θ₃)]
Row 8: [ 0,  0,  0,  0, -1,  0,  0,  1,  L/2·cos(θ₃)]
Row 9: [ 0,  0,  0,  0,  0,  0,  0,  0,  1]
"""
```

Ct (generic function with 1 method)

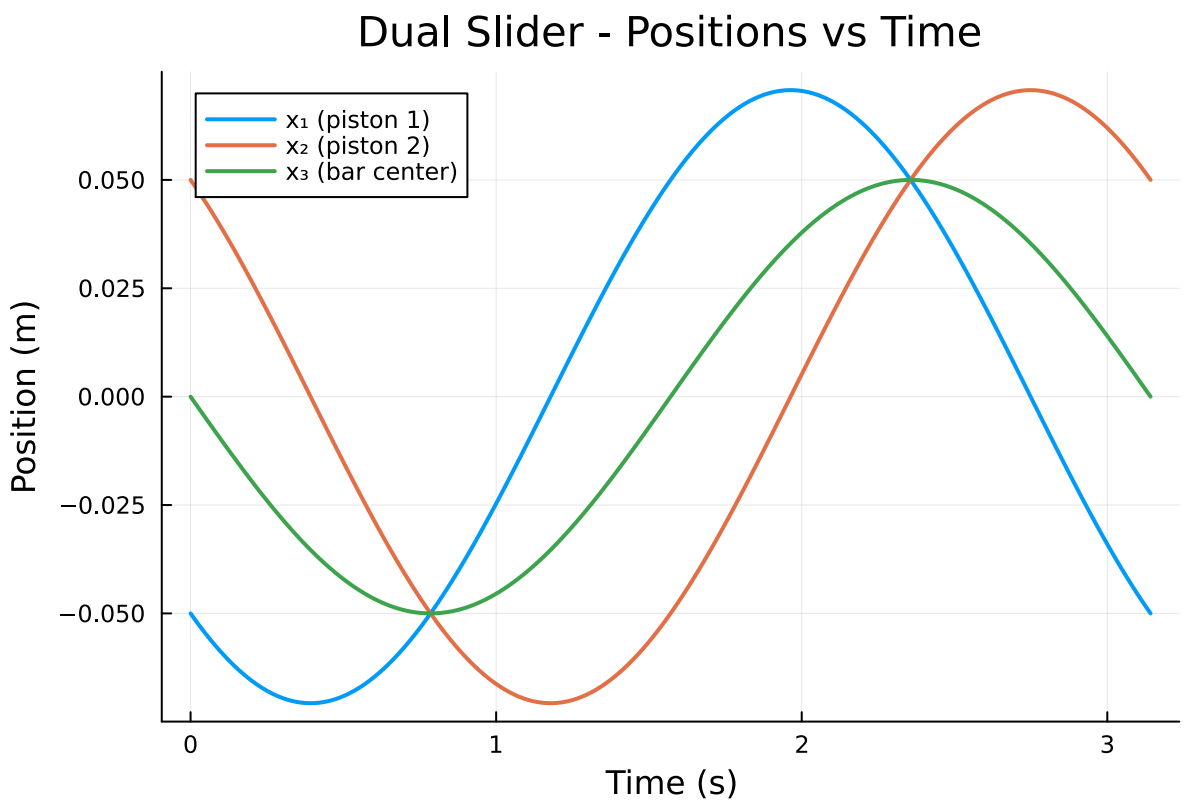
```
begin
    # Jacobian dC/dq (derived by hand)
    function dCdq(q, t)
        x1, y1, θ1, x2, y2, θ2, x3, y3, θ3 = q
        [
            -1   1   0   0   0   0   0   0   0;
             0   0   0   1   1   0   0   0   0;
             0   0   1   0   0   0   0   0   0;
             0   0   0   0   0   1   0   0   0;
            -1   0   0   0   0   0   1   0   L/2*sin(θ3);
             0  -1   0   0   0   0   0   1  -L/2*cos(θ3);
             0   0   0  -1   0   0   1   0  -L/2*sin(θ3);
             0   0   0   0  -1   0   0   1   L/2*cos(θ3);
             0   0   0   0   0   0   0   0   1
        ]
    end

    # Ct = ∂C/∂t — only C9 depends explicitly on time
    function Ct(q, t)
        [0, 0, 0, 0, 0, 0, 0, 0, -2.0]
    end
end
```

```julia
begin
    # Time vector: one full rotation at 2 rad/s takes π seconds
    time = LinRange(0, π, 200)

    # Storage
    q_all  = zeros(length(time), 9)
    dq_all = zeros(length(time), 9)

    # Initial conditions: bar horizontal, pistons on their tracks at origin
    # θ3=0 → bar horizontal → piston 1 at (-L/2·cos45, -L/2·sin45), piston 2 at (+L/2·cos45, -L/2·sin45)
    q0 = [
        -L/2/√2,   # x1
        -L/2/√2,   # y1  (on -45° track: y1 = x1 ✓)
        -π/4,      # θ1
         L/2/√2,   # x2
        -L/2/√2,   # y2  (on +45° track: y2 = -x2 ✓)
         π/4,      # θ2
         0.0,      # x3
         0.0,      # y3
         0.0       # θ3
    ]

    # Solve positions using NonlinearSolve (Newton-Raphson under the hood)
    for i in eachindex(time)
        t = time[i]
        prob = NonlinearProblem((q, p) -> C(q, p), q0, t)
        sol  = solve(prob, NewtonRaphson(), abstol=1e-10)
        q_all[i, :] = sol.u
        q0 = sol.u  # warm start: use previous solution as next guess
    end
end
```

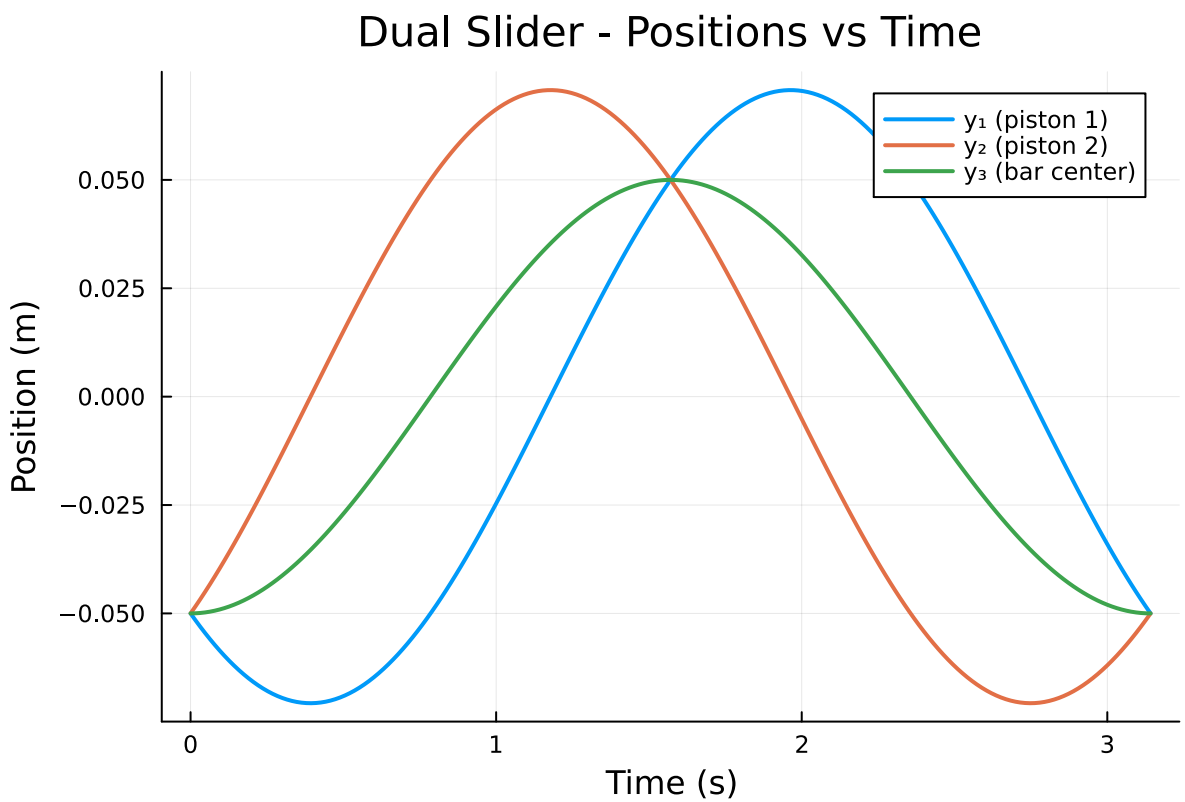
```julia
begin
    # Solve velocities: dCdq * q̇ = -Ct  →  q̇ = dCdq \ (-Ct)
    for i in eachindex(time)
        qt = q_all[i, :]
        dq_all[i, :] = dCdq(qt, time[i]) \ (-Ct(qt, time[i]))
    end
end
```

Position Results

```
md"## Position Results"
```



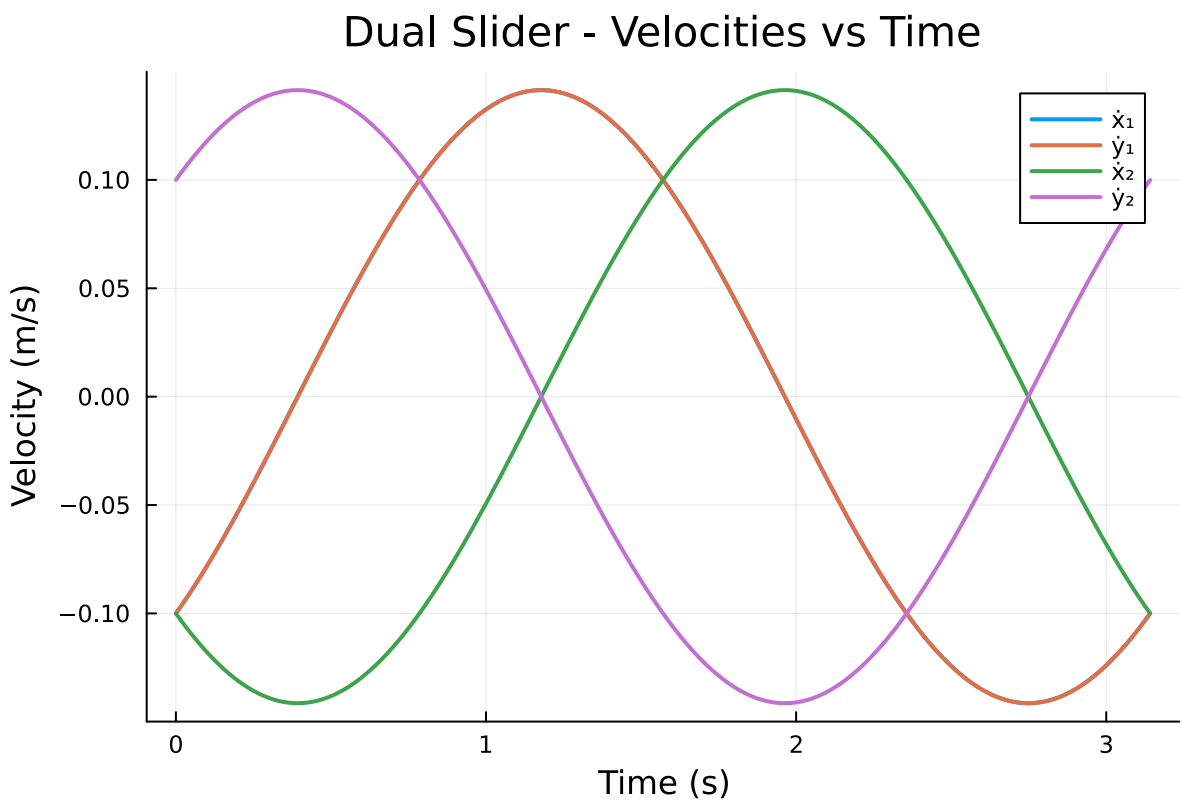

```
begin
    plot(time, q_all[:, 1], label="x₁ (piston 1)", lw=2)

    plot!(time, q_all[:, 4], label="x₂ (piston 2)", lw=2)

    plot!(time, q_all[:, 7], label="x₃ (bar center)", lw=2)

    xlabel!("Time (s)")
    ylabel!("Position (m)")
    title!("Dual Slider - Positions vs Time")
end
```




```
begin
    plot(time, q_all[:, 2], label="y₁ (piston 1)", lw=2)

    plot!(time, q_all[:, 5], label="y₂ (piston 2)", lw=2)

    plot!(time, q_all[:, 8], label="y₃ (bar center)", lw=2)
    xlabel!("Time (s)")
    ylabel!("Position (m)")
    title!("Dual Slider - Positions vs Time")
end
```

Velocity Results

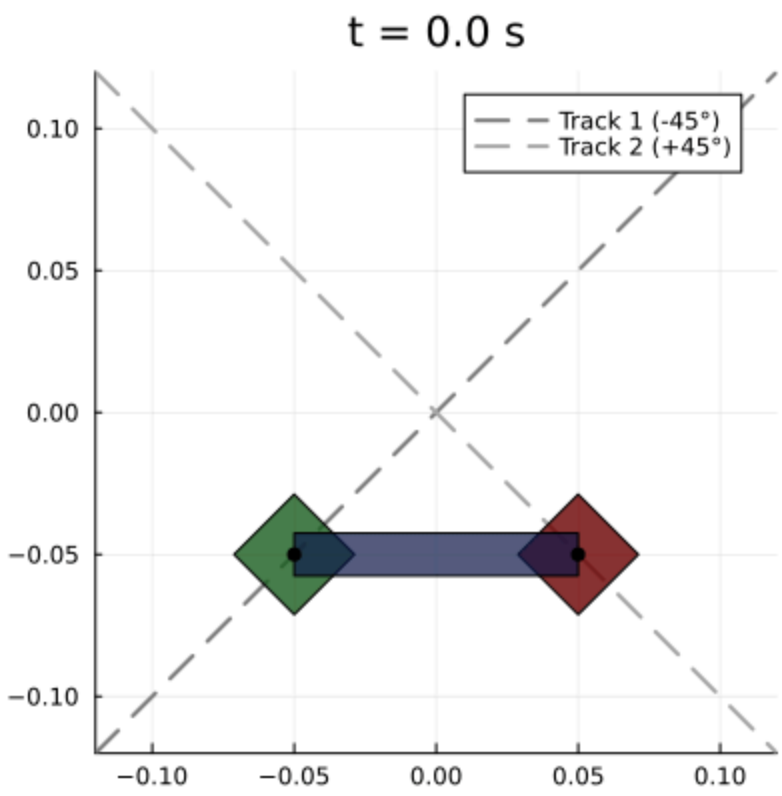
```
md"## Velocity Results"
```




```
begin
    plot(time, dq_all[:, 1], label="ẋ₁", lw=2)
    plot!(time, dq_all[:, 2], label="ẏ₁", lw=2)
    plot!(time, dq_all[:, 4], label="ẋ₂", lw=2)
    plot!(time, dq_all[:, 5], label="ẏ₂", lw=2)
    xlabel!("Time (s)")
    ylabel!("Velocity (m/s)")
    title!("Dual Slider - Velocities vs Time")
end
```

Animation

```
md"## Animation"
```




```julia
begin
    # Helper: draw a rotated rectangle for a rigid body
    function plot_body!(x, y, θ, w, h; color=:neon, alpha=0.5)
        # Corners in local frame
        corners_local = [
            -w/2  -h/2;
             w/2  -h/2;
             w/2   h/2;
            -w/2   h/2;
            -w/2  -h/2
        ]
        R = [cos(θ) -sin(θ); sin(θ) cos(θ)]
        corners_global = (R * corners_local')' .+ [x y]
        plot!(corners_global[:, 1], corners_global[:, 2],
              seriestype=:shape, fillcolor=color, fillalpha=alpha,
              linecolor=:black, label=false)
    end

```

```julia
    # Draw tracks
    function draw_tracks!(; lim=0.15)
        plot!([-lim, lim], [-lim, lim], color=:gray, lw=2, ls=:dash, label="Track 1 (-45°)")
        plot!([-lim, lim], [lim, -lim], color=:darkgray, lw=2, ls=:dash, label="Track 2 (+45°)")
    end

    anim = @animate for i in 1:length(time)
        q = q_all[i, :]
        x1, y1, θ1 = q[1], q[2], q[3]
        x2, y2, θ2 = q[4], q[5], q[6]
        x3, y3, θ3 = q[7], q[8], q[9]

        lim = 0.12
        plot(xlim=(-lim, lim), ylim=(-lim, lim),
             aspect_ratio=:equal, title="t = $(round(time[i], digits=2)) s",
             legend=:topright)

        draw_tracks!(lim=lim)

        # Draw pistons (small squares)
        plot_body!(x1, y1, θ1, L*0.3, L*0.3; color="#1B5E20", alpha=0.8)
        plot_body!(x2, y2, θ2, L*0.3, L*0.3; color="#6D0000", alpha=0.8)

        # Draw bar (long thin rectangle)
        plot_body!(x3, y3, θ3, L, L*0.15; color="#0D1B4B", alpha=0.7)

        # Draw pin joints
        scatter!([x1, x2], [y1, y2], color=:black, ms=4, label=false)
    end

    gif(anim, "dual_slider.gif", fps=30)
end
```

```
Saved animation to C:\Users\cjlai\.julia\pluto_notebooks\dual_slider.gif
```

```
PLUTO_PROJECT_TOML_CONTENTS =
"[deps]\nNonlinearSolve = \"8913a72c-1f9b-4ce2-8d82-65094dcecaec\"\nPlots = \"91a5bcdd-55d7-
```

```
PLUTO_PROJECT_TOML_CONTENTS = """
[deps]
NonlinearSolve = "8913a72c-1f9b-4ce2-8d82-65094dcecaec"
Plots = "91a5bcdd-55d7-5caf-9e0b-520d859cae80"
LinearAlgebra = "37e2e46d-f89d-539d-b4ee-838fcccc9c8e"
"""
```